MegaRTOS

Preemptive Real-Time Operating System
for ATmega328P

System Version: 1.0.1 Stable
Documentation Version: 1.0.1

TECHNICAL DOCUMENTATION
& Reference Manual

Author: Victor Hugo Gamboa Kelly
Embedded Systems Software Developer

21041307@itdurango.edu.mx

Link To Repositories — Author LinkedIn

© 2026 Victor. Distributed under the MIT License.
April 24, 2026

Abstract

MegaRTOS is a preemptive Real-Time Operating System designed for the AVR architecture ATmega328P. The system enables concurrent execution of multiple tasks using a Round Robin scheduling algorithm and a context-switching mechanism optimized in assembly. It provides a robust foundation for applications requiring temporal determinism in memory-constrained environments.

Why use MegaRTOS?

In traditional "Main-Loop" designs, a single delayed function blocks the entire system and wating for I/O too. This leads to jitter and failures in time-critical operations.

Key Advantages:

- **Multitasking:** Run several tasks without LCD delay interference.
- **Non-blocking:** Save clock cycles by suspending tasks.
- **Save energy** it reduce the energy usage when there are no tasks.
- **Scalability:** Add new tasks without breaking existing timing logic.
- **Efficiency:** C++17 and AVR Assembly for a minimal RAM footprint.

Table of Contents

1 Introduction **3**
- 1.0.1 Motivation . . . 3
- 1.0.2 Key Features v1.0.0 . . . 3
- 1.0.3 Target Hardware . . . 3

1.1 Integration Guide . . . 4
1.2 Integrating with Microchip Studio . . . 4
- 1.2.1 Project Configuration . . . 4
- 1.2.2 Compiler Properties (Alt + F7) . . . 5
- 1.2.3 Linker Configuration . . . 6
- 1.2.4 Implementation Notes . . . 6

1.3 Integrating Source Code in Microchip Studio . . . 7
- 1.3.1 Project Structure Integration . . . 7
- 1.3.2 Configuring Assembly Support . . . 7
- 1.3.3 Compiler Settings (Alt + F7) . . . 7
- 1.3.4 Important: Symbols and Linkage . . . 8

2 Basic usage for users **9**
2.1 The Kernel Class . . . 9
- 2.1.1 Access to kernel with the Singleton Pattern . . . 10
- 2.1.2 Core API Reference . . . 10
- 2.1.3 System Inspection (Getters and Setters) . . . 11
- 2.1.4 Code Example: . . . 12

2.2 Resource Synchronization: The Mutex Class . . . 13
- 2.2.1 Mechanism: Lock and Unlock . . . 13
- 2.2.2 Memory Footprint and Overhead . . . 13
- 2.2.3 Basic Usage Example . . . 13
- 2.2.4 Important Considerations . . . 14

2.3 Serial Communication: The UART Driver . . . 15
- 2.3.1 Internal Architecture . . . 15
- 2.3.2 The UART API . . . 15
- 2.3.3 Usage Example . . . 15
- 2.3.4 Technical Summary . . . 16

Chapter 1

Introduction

MegaRTOS is a preemptive Real-Time Operating System (RTOS) designed specifically for the AVR architecture, focusing on the ATmega328P (Arduino Uno). Unlike traditional Arduino development, which often relies on a super-loop architecture, megaRTOS provides a robust framework for multitasking, allowing multiple independent processes to share CPU time efficiently.

1.0.1 Motivation

The primary goal of this project is to apply modern C++ standards (*C++17*) and Low-Level Assembly programming to create a lightweight, predictable, and educational operating system. megaRTOS bridges the gap between high-level abstractions and bare-metal performance, making it ideal for embedded systems where timing and resource management are critical.

1.0.2 Key Features v1.0.0

- **Preemptive Scheduling:** Implements a Round-Robin algorithm driven by hardware timers.
- **Object-Oriented Design:** Uses a Singleton Kernel pattern to manage system resources.
- **Context Switching:** Leverages custom AVR Assembly routines for high-speed task transitions.
- **Synchronization Primitives:** Includes Mutexes and Critical Sections to prevent race conditions.
- **Task Management:** Supports independent stack allocation for each process.

1.0.3 Target Hardware

The system is optimized for the **Microchip ATmega328P**, utilizing its 8-bit AVR architecture, 32KB of Flash, and 2KB of SRAM. By managing the stack and registers manually, megaRTOS ensures that even with limited memory, complex multitasking is achievable.

1.1 Integration Guide

In this section we will cover how to integrate the library into the Microchip Studio IDE, either pre-compiled or by integrating the code directly step by step.

1.2 Integrating with Microchip Studio

To use **megaRTOS** in a professional environment like Microchip Studio, follow these steps to integrate the precompiled library (compiled for the *avr5* architecture).

1.2.1 Project Configuration

1. **Download:** Download the .zip with lib and include folders (Is in Github Release).

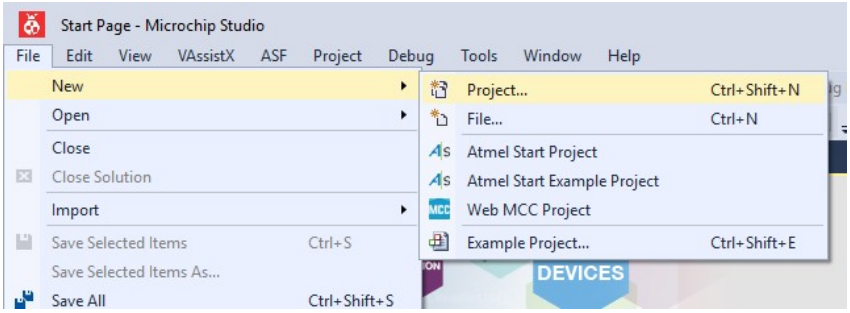

Figure 1.1: Create project in Microchip Studio.

2. **Project Type:** Create a *GCC C++ Executable Project*. This is crucial to ensure `avr-g++` support for modern C++ features.

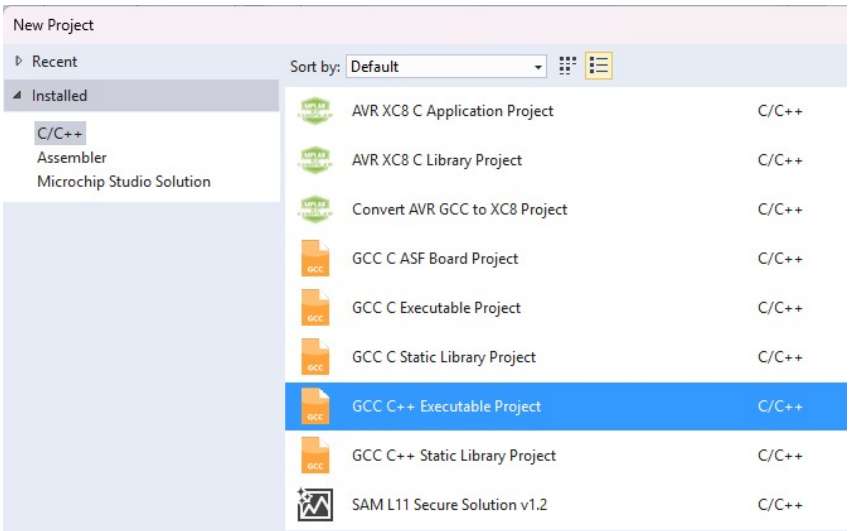

Figure 1.2: GCC C++ Executable Project in Microchip Studio.

3. **Hardware Setup:** Set the target device to **ATmega328P**.

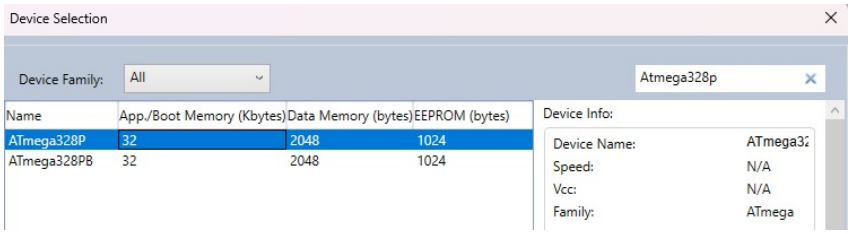

Figure 1.3: Select microcontroller ATmega328P.

4. **Clock Speed:** Ensure that `F_CPU` matches your hardware (e.g., `16000000L` for an Arduino Uno) if you are using 16MHz lib.

5. **Copy Source Files:** Copy the `src/` and `include/` directories from the repository into your project folder.

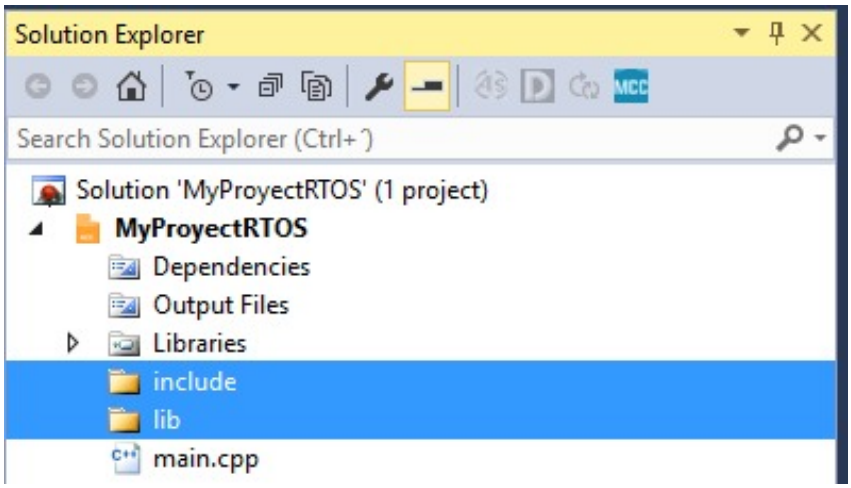

Figure 1.4: Recommended project structure.

1.2.2 Compiler Properties (Alt + F7)

Access the project properties, go to Toolchain and configure the following sections under **AVR/GNU C++ Compiler**:

Directories: Go to Directories- Add `../include` and `../include/kernel` to the search path, Right-click on the include folder and select copy the full path, then click on the green directory table icon and paste the address, then paste the path again adding kernel to the end of the path..

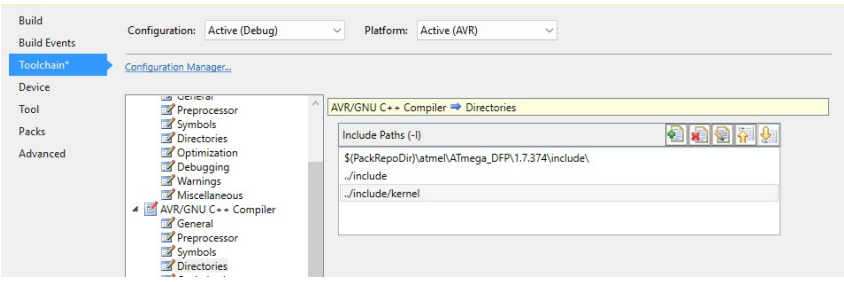

Figure 1.5: Directories Required in project.

Optimization: Go to Optimization- Set the optimization level to *Optimize for size* (`-Os`) to minimize Flash usage.

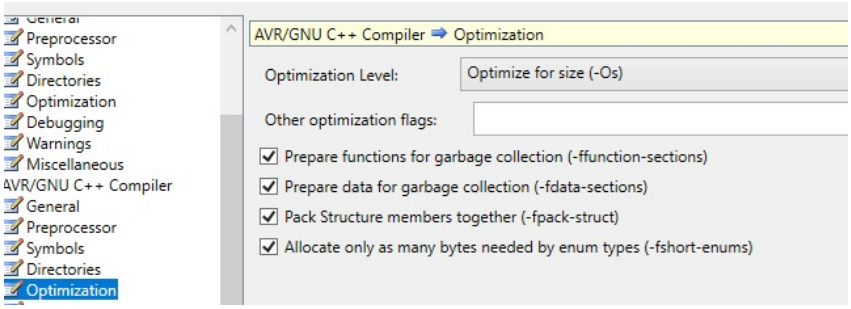

Figure 1.6: Level of optimization.

Language Standard: In the *Miscellaneous*- flag section, add the flag `-std=c++17` to enable modern C++ features.

Figure 1.7: Directories Required in project.

1.2.3 Linker Configuration

Navigate to **AVR/GNU Linker** to link the static library:

Libraries (-l): Go to Libraries- Add `megaRTOS`. Note: Omit the `lib` prefix and the `.a` extension.

Search Path (-L): Add the path to your `/lib` folder where the file `libmegaRTOS.a` is located.

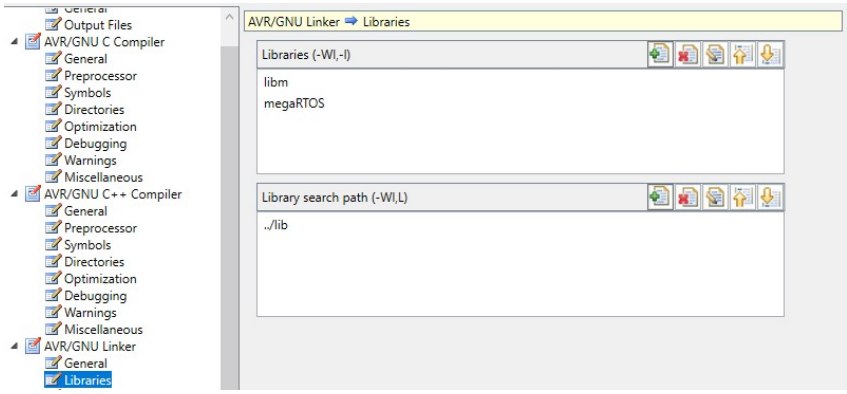

Figure 1.8: Linker libraries configuration.

1.2.4 Implementation Notes

- **File Extension:** Ensure your entry point file is named `main.cpp`.
- **Header Compatibility:** Use `#include <avr/io.h>` instead of `xc.h` to maintain full C++ compatibility and avoid linkage issues with the kernel.

1.3 Integrating Source Code in Microchip Studio

If you prefer to include the **megaRTOS** source code directly into your Microchip Studio project instead of using a precompiled library, follow these instructions. If you want to compile your project at a different frequency than the pre-compiled ones, or if you want to modify the kernel source code, you can implement it as follows.

1.3.1 Project Structure Integration

1. **Copy Source Files:** Copy the `src/` and `include/` directories from the repository into your project folder.

2. **Add to Solution:** In the *Solution Explorer*, right-click your project and select **Add ¿ Existing Item...** to include all `.cpp`, `.hpp`, and the `.S` files.

1.3.2 Configuring Assembly Support

Since megaRTOS relies on custom assembly, ensure the file `switchContextASM.S` is present. The compiler will automatically invoke `avr-gcc` for files with the capital `.S` extension.

1.3.3 Compiler Settings (Alt + F7)

Under **AVR/GNU C++ Compiler**:

Include Paths: Add paths to `include` and `include/kernel`.

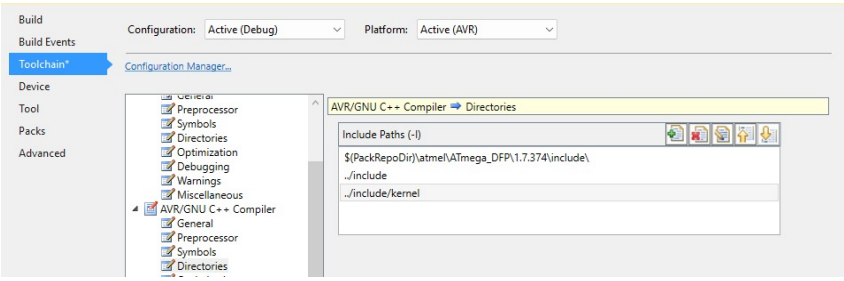

Figure 1.9: Directories Required in project.

Language Standard: Ensure you have the flag `-std=c++17`.

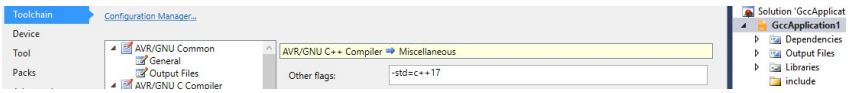

Figure 1.10: Cpp version required.

Optimization: It is recommended to use `-Os` (Optimize for size).

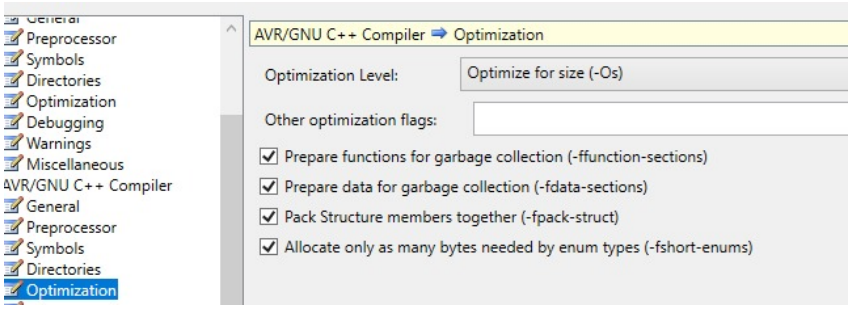

Figure 1.11: Directories Required in project.

1.3.4 Important: Symbols and Linkage

When compiling from source, ensure that:

- **No Library Linking:** Remove `megaRTOS` from *Linker - Libraries*.
- **Interrupt Handlers:** Ensure that the `ISR(TIMER0_COMPA_vect)` is only defined once.

Chapter 2

Basic usage for users

In this chapter we will explore the available functions of the megaRTOS user interface classes and their general purpose, as well as recommendations for the current version. This section is aimed at people who only want to implement the library and its utilities; for people who want to know the code to modify or adapt it, check the code section

2.1 The Kernel Class

The `Kernel` class is the central component of **megaRTOS**. It manages the lifecycle of all processes and controls the system's execution flow. Essentially, the task of the kernel class is to enable functions for the user and initialize the system hardware.

```
// To use the class you need include Kernel header
#include "include/kernel/Kernel.hpp"
```

Figure 2.1 shows the basic flow diagram of the system to understand the general behavior

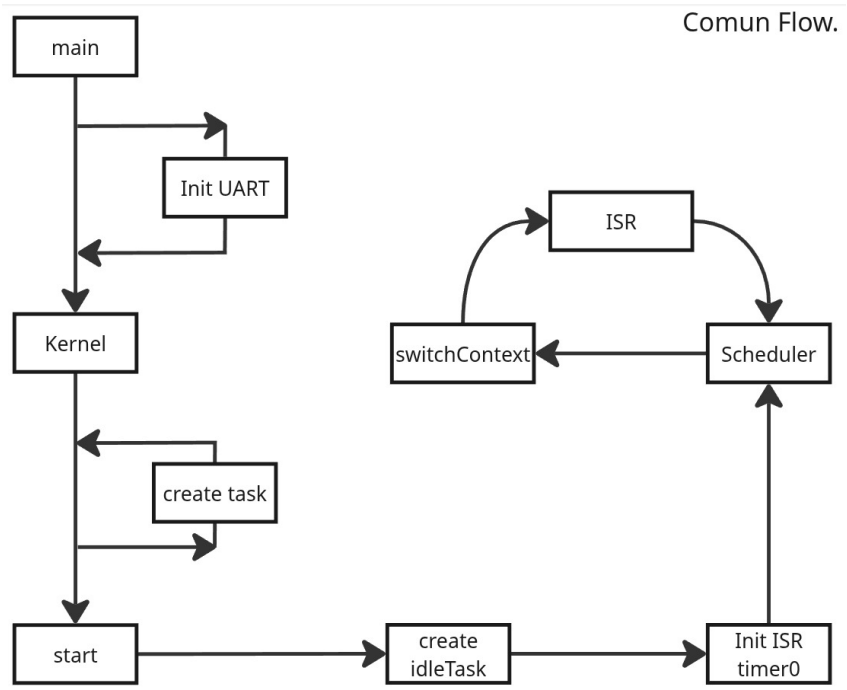

Figure 2.1: Conceptual flow of the megaRTOS Kernel and its internal state management.

2.1.1 Access to kernel with the Singleton Pattern

To ensure system stability, the Kernel is implemented as a *Singleton*. This means there is only one global instance of the operating system running. Users cannot instantiate the class directly to avoid double instantiation of the class, thus avoiding unnecessary memory usage as well as undefined behaviors in the system.

- **Usage:** To access any Kernel function, you must use the `getInstance()` method.
- **Note:** The function to obtain the Kernel class instance is static, and the object is also static. Keep in mind that simply compiling the code with the included RTOS occupies memory on the microcontroller.

```
// Accessing the Kernel instance
Kernel& myKernel = Kernel::getInstance();
```

By using kernel class functions accessed directly from the instance without creating a reference to it, such as '//Kernel::getInstance().delay(ms);' , you can implement the facade pattern by creating short functions that do the work. Note: megaRTOS does not implement this by default because it would be redundant, although it is implemented in other classes within the system.

```
// Facade example
void osDelay(uint16_t ms){
    Kernel::getInstance.delay(ms);
}

void task1(){
    while(1){
        //your code...
        osDelay(500);
    }
}

int main(){
    // kernel access by reference (recommended in main).
    Kernel& os = Kernel::getInstance();

    os.createTask(task1, 1);

    os.start();

    return 0;
}
```

2.1.2 Core API Reference

Function: createTask();

Registers a new function as a managed task within the RTOS. This function creates an object of the Process class, which represents a process within the system that will be managed by the kernel, eventually assigning it CPU time once it has started.

- **Purpose:** To add a specific behavior (task) to the execution queue.
- **Parameters:**
  - `taskFunction`: A pointer to a function that returns `void` and takes no arguments.
  - `priority`: An integer value (higher values represent higher priority) Priority is not currently used within the system, but its use is planned for future versions of priority-based scheduling algorithms.
- **specifications**
  - `Size`: Each process needs approximately 140 bytes of memory.
  - `Cuantity`: The system handles up to 7 user tasks in version 1.0.1 and an idle task or daemon that runs when there are no tasks that require CPU time.
  - `PID`: Each process has a PID and it is the time it arrives in the process queue; the first task created has id 1.
  - `Note`: For more technical information about the classes, please review the technical section of the documentation.

Function: delay();

Suspends the execution of the calling task blocking it.

- **Purpose:** To pause a task for a specific amount of time without wasting CPU cycles, allowing other tasks to run.
- **Parameter:** `ms` (milliseconds).
- **Note:** This function puts the task it is called into to sleep. Calling this function outside of a task may put the process currently on the CPU to sleep or cause undefined behavior.
- **Note:** There is only a delay in milliseconds; it will probably be implemented for other parameters in future versions.

Function: start();

Transfers control to the RTOS.

- **Purpose:** This function initializes the hardware timers and starts the scheduler. It is typically called at the end of the `main()` function. Once called, this function never returns.

2.1.3 System Inspection (Getters and Setters)

Getter: getCurrentProcess();

- **Purpose:** Returns a pointer to the `Process` object that is currently executing. Useful for debugging or advanced task management.

Getter: getProcessCount();

- **Purpose:** Returns the total number of tasks currently registered in the system.

Setter: setCurrentProcess();

- **Warning:** *Manual use of this function is strongly discouraged.*
- **Purpose:** Manually sets the pointer to the active process. This is primarily used by synchronization primitives (like Mutexes) and internal kernel routines. Improper use can lead to immediate system crashes.

2.1.4 Code Example:

Here is a basic snippet of how a user interacts with the Kernel API:

```
// Include header
#include "yourRute/kernel/Kernel.hpp"

void osDelay(uint16_t ms){
    kernel::getInstance().delay(ms);
}

// Process 1
void myTask1(){
    while(true){
        //task logic here
        osDelay(500);
    }
}

// Process 2
void myTask2() {
    while(true) {
        // Task logic here
        osDelay(250);
    }
}

// Main
int main() {
    // Accessing to Kernel class using reference
    Kernel& os = Kernel::getInstance();

    // We create process number 1 and 2
    os.createTask(myTask1, 1);
    os.createTask(myTask2, 1);

    // Start RTOS and gives the flow
    os.start()
```

```
    // Flow never gets here
    return 0;
}
```

2.2 Resource Synchronization: The Mutex Class

In a preemptive multitasking environment like **megaRTOS**, multiple tasks may attempt to access the same hardware resource (e.g., a shared UART bus or a global variable) simultaneously. The `Mutex` class provides a mutual exclusion primitive to prevent data races and ensure system integrity.

2.2.1 Mechanism: Lock and Unlock

The Mutex works as a binary semaphore with an ownership concept. It implements a non-busy-waiting queue, meaning that tasks waiting for a resource do not consume CPU cycles.

- **lock()**: If the Mutex is free, the calling task becomes the owner. If it is already locked by another task, the calling task is automatically moved to the `WAITING_MUTEX` state and added to a linked list. The scheduler then switches to the next `READY` task.
- **unlock()**: The current owner releases the resource. If there are tasks waiting in the queue, the Kernel wakes up the next process in line, moving it back to the `READY` state.

2.2.2 Memory Footprint and Overhead

The `Mutex` class is designed to be extremely lightweight, which is critical for the ATmega328P's limited 2KB of SRAM.

- **RAM Usage:** Each Mutex object only requires two pointers (`m_owner` and `m_waitListHead`). On an 8-bit AVR architecture, this translates to only **4 bytes** of memory per instance.
- **Scalability:** The user can create multiple Mutex objects to protect different resources independently.

2.2.3 Basic Usage Example

It is recommended to define Mutexes globally or as shared resources between tasks.

```
// Header needed
#include "include/Kernel/Mutex.hpp"

Mutex serialMutex;

// Print AAAAAA\r\n in serial port
```

```
void taskPrintA() {
    while (true) {
        serialMutex.lock();   // Request UART ownership
            UART_print_str("AAAAAA\r\n");
        serialMutex.unlock(); // Release UART ownership
        Kernel::getInstance().delay(15);
    }
}

// Print BBBBBB\r\n in serial port
void taskPrintB() {
    while (true) {
        serialMutex.lock();
            UART_print_str("BBBBBB\r\n");
        serialMutex.unlock();
        Kernel::getInstance().delay(15);
    }
}

int main(){
    // Kernel access by reference
    Kernel os = Kernel::getInstance();

    os.createTask(taskPrintA, 1);
    os.createTask(taskPrintA, 1);

    os.start();

    // Never gets here
    return 0;
}

// If you do not use mutex in this example the output could be similar to ABABABA
// With Mutex
// AAAAAA
// BBBBBB
```

2.2.4 Important Considerations

Efficiency: When a task is blocked by a Mutex, it is completely ignored by the scheduler. This ensures that 100% of CPU time is dedicated to tasks that can actually perform work.

Deadlocks: Users must ensure that every `lock()` call has a corresponding `unlock()`. Additionally, avoid locking multiple Mutexes in different orders across tasks to prevent circular wait conditions.

2.3 Serial Communication: The UART Driver

The `UART` class manages the asynchronous serial communication for the ATmega328P. Unlike standard Arduino `Serial` implementations, this driver is tightly integrated with the **megaRTOS** scheduler, allowing tasks to yield the CPU while waiting for data.

2.3.1 Internal Architecture

The driver uses a Producer-Consumer model:

- **Producer:** The Hardware Interrupt Service Routine (ISR) receives bytes and places them in an internal **Circular Buffer** of 32 bytes.
- **Consumer:** The user task requests data. If the buffer is empty, the task is moved to a `WAITING` state, and the Kernel resumes it only when new data arrives.

2.3.2 The UART API

The library provides high-level wrappers to simplify the syntax, avoiding the need to call `getInstance()` repeatedly.

Initialization

- `UART_init()`: Configures the UART peripheral. By default, it is optimized for 9600 baud (or the rate defined in your `Config.hpp`). It enables both Transmitter and Receiver with RX-complete interrupts.

Reading Data (Non-Busy Waiting)

- `UART_read_blocking()`: This is the most efficient way to receive data.
- **Behavior:** If the buffer has data, it returns the character immediately. If not, it **blocks the current process**. The CPU will not execute a single line of this task until a byte physically arrives at the RX pin.

Transmission

- `UART_write(char c)`: Sends a single character through the TX line.
- `UART_print_str(const char* string)`: Transmits a null-terminated string.

2.3.3 Usage Example

This example demonstrates a task that waits for user input without halting other system tasks.

```
#include "include/Kernel/UART.hpp"

//
void commandTask() {
    UART_init();
    UART_print_str("System Ready. Press any key...\n");

    while(true) {
        char c = UART_read_blocking(); // Task blocks here

        UART_print_str("Received: ");
        UART_write(c);
        UART_write('\n');
    }
}
```

2.3.4 Technical Summary

Component	**Value/Behavior**
Buffer Type	Circular (Thread-safe)
Buffer Size	32 Bytes
Wait Mechanism	Context Switch (No-Poling)
Memory Usage	$\approx$ 36 bytes (Buffer + Pointers)

Table 2.1: UART Driver Specifications.